

Metodología de Sistemas I

Tecnicatura Universitaria en Programación

Ing. Landoni, Hernán

Clase 1

Bienvenida e introducción a la materia

Bloque 1

Contexto Macro, Fundamentos, Requerimientos, UML, Arquitectura

Tipos de software

Para simplificar, podemos dividir al software en 3 grandes grupos con los que interactuamos de forma regular:

De sistema o de base

Sirve de soporte o base para controlar e interactuar con el hardware y otros programas.

Ejemplos: Sistemas operativos, controladores, sistemas embebidos, etc.

De programación

Herramientas que permiten al programador desarrollar software usando lenguajes de programación, de una manera práctica.

Ejemplos: Editores de texto, compiladores, IDEs, etc.

De aplicación

Permite a los usuarios realizar tareas específicas, especialmente en el ámbito de negocios, mediante automatización o asistencia.

Ejemplos: Bases de datos, programas de CAD, Videojuegos, Aplicaciones Ofimáticas, ERPs, etc.

Tipos de aplicaciones

Dentro del software de aplicación, podemos también categorizarlo en 3 grandes grupos

Desktop	Web	Mobile
Instalado en la computadora. Generalmente ofrecen acceso offline. Word · Visual Studio · Autocad	Corre en el navegador. Accesible desde cualquier dispositivo con internet. Gmail · MercadoLibre · Figma · Notion	Apps nativas para Android/iOS. Interfaz táctil, notificaciones push. WhatsApp · Instagram · Uber
Menos común hoy	Más común hoy	Complementa lo web

El software que nos rodea

Todo empezó como un problema que alguien quiso resolver...



**mercado
libre**



despegar



Discord



¿Por qué nace un proyecto de software?



Problema

Alguien tiene una necesidad insatisfecha o un proceso ineficiente

Ej: el almacén lleva todo en papel y pierde ventas



Solución

Se decide construir o comprar un sistema que resuelva ese problema

Ej: una app web para registrar ventas e inventario



Producto

El software terminado que entrega valor real al usuario

Ej: MercadoLibre, Pedidos Ya, el sistema de turnos del hospital

La metodología define CÓMO vamos del problema al producto de la manera más eficiente posible.

Tipos de organizaciones de software

Tipo de organización	Funciones	Equipos	Ejemplos	¿Qué exigen del programador?
Corporaciones	Grandes empresas con producto propio.	Grandes, con roles definidos.	E-commerce, Bancos, Aseguradoras, etc.	• Calidad del código • Documentación • Procesos
Consultoras	Empresas que desarrollan software para otras.	Varían con el proyecto, con roles definidos.	Software Factories de diferentes tamaños.	• Adaptabilidad • Comunicación • Entregar a tiempo
PyMEs	Empresas de cualquier otro rubro con áreas de IT.	Pocas personas, cada una ocupa varios roles.	Talleres metalmecánicos, siderúrgicas, empresas logísticas, etc.	• Adaptabilidad • Tolerar incertidumbre • Autonomía
Startups	Pequeñas empresa con producto propio y tiempo muy limitado.	Pocas personas, cada una ocupa varios roles.	Fintech, Insurtech, Traveltech, Agtech, etc.	• Velocidad • Visión de producto • Adaptabilidad
Sector público	Dar servicio a los ciudadanos.	Varían con el proyecto.	Hospitales públicos, Ministerios, Municipios, etc.	• Rigor y normativa • Documentación • Trabajo a largo plazo
Freelance	Emprendimientos eventuales o sostenidos.	Una sola persona o con muy pocos compañeros.	Automatizaciones de procesos, desarrollo de webs comerciales, etc.	• Visión de negocio • Tolerar incertidumbre • Cubrir todo

El rol del programador en las organizaciones

Construye

Escribe el código que hace funcionar el producto

Decide

Elige tecnologías, arquitecturas y soluciones técnicas

Colabora

Trabaja con diseñadores, PMs, QA y otros devs

Entiende

Lee el contexto del negocio para construir lo correcto

Comunica

Explica decisiones técnicas a personas no técnicas

*El programador que solo
construye
es reemplazable.*

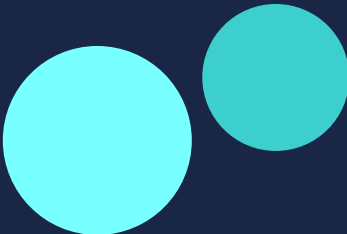
*El que también entiende
por qué construye,
no lo es.*

¿Qué sistema usarías como proyecto integrador?

Un sistema que ya existe

Algo que me gustaría que existiera

Todavía no sé



Clase 2

El proyecto de software: tipos, etapas y ciclo de vida

Recordemos, ¿Por qué nace un proyecto de software?



Problema

Alguien tiene una necesidad insatisfecha o un proceso ineficiente

Ej: el almacén lleva todo en papel y pierde ventas



Solución

Se decide construir o comprar un sistema que resuelva ese problema

Ej: una app web para registrar ventas e inventario

¿Qué orígenes concretos tienen los proyectos de software?

Origen	Descripción	Pregunta Clave	Riesgos
Problema concreto	Alguien tiene una necesidad insatisfecha o un proceso que falla.	¿Qué duele más?	El problema llegan con una solución previamente planteada que contamina los requerimientos.
Oportunidad de negocio	Existe un mercado o necesidad que nadie resolvió todavía.	¿Qué falta en el mercado?	Al ser algo desconocido los requerimientos cambian constantemente.
Decisión estratégica	Una organización elige construir en lugar de depender de terceros.	¿Qué conviene controlar del proceso?	Pocos, los requerimientos ya están claros porque se requiere algo similar a lo que ya se tiene.

Del problema al código: algo se pierde en el camino



La mayoría de los problemas en proyectos de software tienen origen en comunicación, no en código.

Proyecto vs. producto

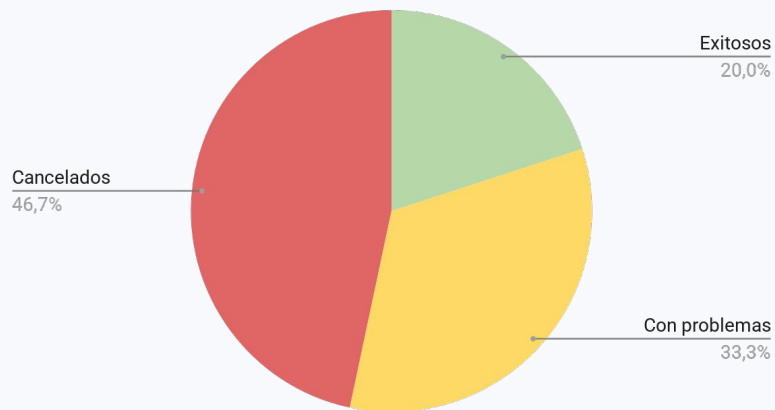


No todos los proyectos son iguales...

Tipo	Descripción
Greenfield	Se construye desde cero, sin sistema previo
Mantenimiento y evolución	El sistema existe y se mejora sin romper lo que funciona
Migración tecnológica	La funcionalidad existe pero se traslada a una nueva plataforma
Integración entre sistemas	Dos o más sistemas independientes necesitan comunicarse
Reingeniería o rescate	Un proyecto fallido o en riesgo que se diagnostica y replantea

¿Qué tan seguido fallan los proyectos de software?

Distribución de resultados



Causas comunes

- Requerimientos incompletos o cambiantes
- Falta de participación del usuario
- Expectativas no alineadas entre cliente y equipo
- Cambios de alcance sin control

Fuente: Standish Group, Chaos Report. [\(1\)](#) [\(2\)](#)

¿Cómo se estructuran los proyectos de software?

Software Development Life Cycle

(Ciclo de vida del desarrollo de software)

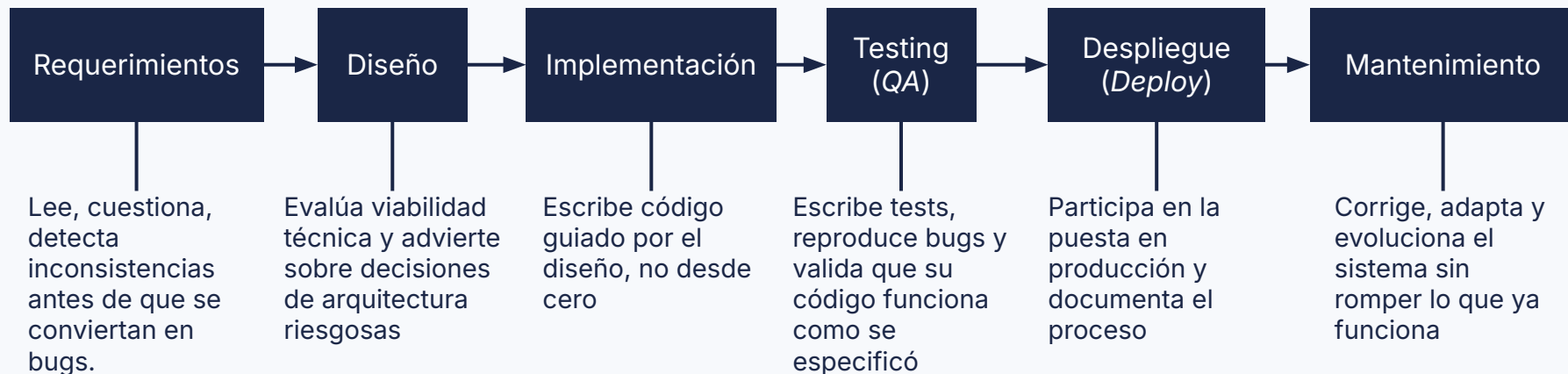


Todas las metodologías recorren estas seis etapas

Ejemplo: App de turnos médicos.

Tipo de organización	Qué se hace	Quién	Qué produce
Requerimientos	Se entrevista al médico, recepcionista y pacientes. Se documenta qué debe hacer el sistema.	Analista funcional	Documento de requerimientos
Diseño	Se define la arquitectura, el modelo de datos y las interfaces	Arquitecto / Dev senior	Documento de Diseño Técnico
Implementación	Se escribe el código guiado por el diseño	Equipo de desarrollo	Código fuente + documentación
Testing	Se verifica que el sistema funciona y no tiene bugs críticos	QA + equipo de desarrollo	Plan de pruebas + reporte de defectos
Despliegue	Se instala en producción y se capacita a los usuarios	DevOps / equipo técnico	Release notes + documentación de usuario
Mantenimiento	Se corrigen bugs, se adapta a cambios y se agregan funcionalidades	Equipo de desarrollo	Nuevas versiones del sistema

El programador en el SDLC



El programador que solo codea ejecuta instrucciones. El que entiende el ciclo completo toma decisiones.

Actividad

Escenario 1:

Un municipio contrató una consultora para desarrollar un sistema de gestión de reclamos ciudadanos. Seis meses después del lanzamiento, el sistema está en producción pero los empleados no lo usan porque el flujo de trabajo no coincide con cómo operan internamente.

Escenario 2:

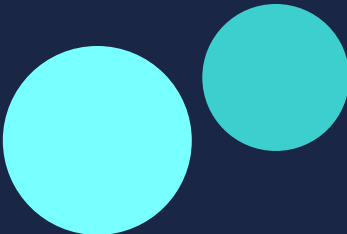
Una clínica tenía un sistema de turnos de 2008 sin soporte. Se contrató un equipo para migrarlo a tecnología actual. La migración se completó en tiempo y presupuesto, pero se perdieron tres meses de historial de turnos porque nadie documentó el esquema de datos del sistema viejo.

Escenario 3:

Una fintech necesitaba integrar su plataforma con tres bancos locales para procesar transferencias. El equipo estimó dos meses. A los cuatro meses, solo un banco está integrado: cada banco tiene una API distinta con formatos de datos incompatibles, y nadie lo había relevado antes de arrancar.

Responder:

1. ¿Qué tipo de proyecto es?
2. ¿Qué etapas del SDLC creés que se ejecutaron mal?
3. ¿En qué etapa se originó el problema principal?



Clase 3

SDLC en profundidad, artefactos, roles y stakeholders

Recordemos:



Todas las metodologías recorren estas seis etapas

**Vamos a usar la app de turnos que
construimos anteriormente**

Antes de avanzar, una nueva definición:

Artefacto:

Un artefacto es un **producto tangible resultante del proceso de desarrollo de software**.

Algunos artefactos como los casos de uso, diagrama de clases u otros modelos UML **ayudan a la descripción** de la función, la arquitectura o el diseño del software.

Otros **se enfocan en el proceso de desarrollo en sí mismo**, como planes de proyecto, casos de negocios o enfoque de riesgos.

Habitualmente se refiere a la **documentación generada a lo largo del desarrollo** del producto en lugar del producto en sí.

Requerimientos



¿Qué se hace?	¿Quién lo hace?
Se entrevista al médico, la recepcionista y los pacientes. Se documenta qué debe hacer el sistema.	Analista funcional, con participación del cliente y usuarios finales.
¿Qué produce?	¿Qué pasa si se hace mal?
Documento de requerimientos, lista de funcionalidades, restricciones y criterios de aceptación.	Se construye el sistema equivocado. El error se descubre en testing o en producción, cuando corregirlo cuesta mucho más.

¿De qué se trata?

Convertir el problema del cliente en algo que un equipo técnico pueda construir.

¿Qué se hace?

- Entrevistas al cliente y usuarios finales
- Observación del flujo de trabajo actual
- Talleres de relevamiento con todos los stakeholders
- Análisis de documentación existente
- Definición del alcance del sistema

Requerimientos: artefacto y errores típicos



Artefacto:

Especificación de requisitos de software (ERS) o SRS (Software Requirements Specification)

Documento que define qué debe hacer el sistema, bajo qué condiciones y con qué restricciones. Es el contrato entre el negocio y el equipo técnico.

Todo lo que no está en ese documento no existe para el equipo técnico. Todo lo que está mal escrito en ese documento va a generar problemas en implementación.

Errores típicos:

- Requerimientos ambiguos: cada persona los interpreta distinto
- Requerimientos implícitos: el cliente asume que son obvios y no los menciona
- Requerimientos contradictorios: dos stakeholders piden cosas incompatibles
- Scope creep: el alcance crece durante el relevamiento sin control ni priorización

Diseño



¿Qué se hace?	¿Quién lo hace?
Se define cómo se estructura la app: base de datos de pacientes y turnos, pantallas del sistema, integración con WhatsApp para recordatorios.	Arquitecto o desarrollador senior, con validación del analista.
¿Qué produce?	¿Qué pasa si se hace mal?

Documento de Diseño Técnico: modelo de datos, diagrama de arquitectura, wireframes de las pantallas principales.

Se elige una base de datos que no soporta consultas concurrentes. En el lanzamiento, dos recepcionistas asignan el mismo turno al mismo horario.

¿De qué se trata?

Definir cómo va a estar construido el sistema antes de escribir una línea de código.

¿Qué se hace?

- Diseño de arquitectura del sistema
- Modelado de datos: entidades, relaciones y estructura de la base de datos
- Definición de interfaces de usuario y de APIs
- Decisiones de tecnología: lenguaje, frameworks, infraestructura
- Estimación de tiempos y recursos en base al diseño

Diseño: artefacto y errores típicos



Artefacto:

Documento de Diseño Técnico

Define la arquitectura del sistema, el modelo de datos, las interfaces y las decisiones tecnológicas. Es la guía que el equipo de desarrollo va a seguir durante la implementación.

Es el documento que el equipo de desarrollo va a consultar durante la implementación cada vez que tenga una duda sobre cómo está pensado el sistema.

Errores típicos:

- Over-engineering: diseñar para una escala o complejidad que el sistema nunca va a necesitar
- Under-engineering: diseñar sin considerar el crecimiento futuro del sistema
- Decisiones tecnológicas por moda: elegir una tecnología por ser nueva, no por ser la correcta para el problema
- Diseño sin validación: nunca se confirma con el cliente que el diseño resuelve el problema original

Implementación



¿Qué se hace?	¿Quién lo hace?
Se escribe el código: el módulo de turnos, el panel de la recepcionista, el sistema de notificaciones, la vista del paciente.	Equipo de desarrollo, guiado por el documento de diseño.
¿Qué produce?	¿Qué pasa si se hace mal?
Código fuente versionado y documentación técnica del sistema.	Cada desarrollador interpreta el diseño a su manera. El módulo de turnos y el de notificaciones no se integran correctamente.

¿De qué se trata?

Construir lo que se diseñó, no lo que se ocurre en el momento.

¿Qué se hace?

- Escritura de código guiada por el diseño
- Revisiones de código entre pares (code review)
- Documentación técnica del código
- Integración continua de los componentes desarrollados
- Comunicación constante con el equipo ante dudas o bloqueos

Implementación: artefacto y errores típicos



Artefacto:

Código fuente + documentación técnica

El código que implementa las funcionalidades definidas en el diseño, acompañado de documentación que explica decisiones de implementación no evidentes.

Es la documentación que vive cerca del código y explica por qué se tomó una decisión de implementación que no es evidente.

Errores típicos:

- Desviarse del diseño sin documentar el cambio
- Código sin documentación: funciona hoy, nadie lo entiende mañana
- No hacer code review: errores que otro hubiera detectado en 5 minutos llegan a producción
- Deuda técnica silenciosa: soluciones rápidas que resuelven el problema hoy y lo complican mañana

Antes de avanzar, una nueva definición:

Deuda técnica:

La deuda técnica es el **costo futuro** de retrabajo que surge al priorizar la **entrega rápida** de software sobre una mejor solución o diseño, asumiendo "atajos" en el código. Al igual que una deuda financiera, acumula "intereses" (hace más difícil trabajar en el sistema luego) y **debe pagarse mediante refactorización** para evitar problemas de mantenimiento o inestabilidad.

Refactorización:

Técnica de ingeniería de software que **mejora la estructura interna del código fuente** (legibilidad, mantenibilidad) sin alterar su comportamiento externo. Se utiliza para eliminar deuda técnica, reducir la complejidad y facilitar futuros cambios, generalmente ejecutando pruebas unitarias para garantizar que la funcionalidad no cambie.

Testing



¿Qué se hace?	¿Quién lo hace?
Se prueba que el sistema funciona según lo especificado y se valida con el cliente que el flujo resuelve su trabajo diario.	QA ejecuta los tests. La recepcionista y el médico participan en las pruebas de aceptación.
¿Qué produce?	¿Qué pasa si se hace mal?
Plan de pruebas y reporte de defectos. Ej: "al cancelar un turno, el horario no queda disponible para otro paciente".	El sistema pasa todos los tests técnicos pero la recepcionista no puede usarlo porque el flujo no coincide con cómo trabaja realmente.

¿De qué se trata? Confirmar que el sistema funciona correctamente y que resuelve el problema original.

- **Verificación:** ¿construimos el sistema correctamente? (según el diseño y los requerimientos)
- **Validación:** ¿construimos el sistema correcto? (según la necesidad real del cliente)

¿Qué se hace?

- Pruebas unitarias: verificar funciones y componentes de forma aislada
- Pruebas de integración: verificar que los componentes funcionan juntos
- Pruebas de sistema: verificar el sistema completo en un entorno similar a producción
- Pruebas de aceptación: el cliente confirma que el sistema cumple sus expectativas

Testing



Artefacto:

Plan de pruebas + reporte de defectos

El plan de pruebas define qué se va a probar, cómo y bajo qué condiciones. El reporte de defectos documenta los bugs encontrados, su severidad y su estado de resolución.

Un sistema que pasa todas las pruebas técnicas pero no resuelve el problema del cliente no es un éxito. Es un fracaso bien documentado.

Errores típicos:

- Testing como etapa final: se prueba solo al final cuando corregir es más caro
- Pruebas insuficientes: se prueban los casos felices pero no los casos de borde
- Testing solo a cargo del QA: el programador no prueba su propio código
- Validación saltada: se verifica contra los requerimientos pero nunca se confirma con el cliente

Antes de avanzar, una nueva definición:

Casos borde (Edge Cases)

Un problema o situación que se sale de los procedimientos normales. Estas situaciones suelen implicar valores de entrada extremos o comportamientos de usuario poco comunes que, si no se gestionan adecuadamente, pueden provocar fallos en la aplicación o vulnerabilidades de seguridad.

Despliegue



¿Qué se hace?	¿Quién lo hace?
Se instala la app en el servidor de la clínica, se migran los turnos existentes y se capacita a la recepcionista y al médico.	DevOps o equipo técnico.
¿Qué produce?	¿Qué pasa si se hace mal?
Release notes con los cambios de la versión y manual de usuario para la recepcionista.	La recepcionista recibe el sistema el lunes a las 8am sin capacitación. El primer día de uso es un caos y el médico decide volver a la agenda de papel.

¿De qué se trata?

Poner el sistema en producción y asegurar que los usuarios puedan operarlo.

¿Qué se hace?

- Instalación del sistema en el entorno de producción
- Migración de datos del sistema anterior si existe
- **Capacitación a los usuarios finales**
- Comunicación del lanzamiento a los stakeholders
- **Monitoreo del sistema en las primeras horas de operación**

Antes de avanzar, una nueva definición:

Copia de seguridad (Backup)

Una copia de datos digitales almacenada por separado del original para permitir su restauración en caso de pérdida, corrupción o fallo del hardware. Actúa como una medida de seguridad contra la eliminación accidental o desastres.

Reversión de cambios (Rollback)

Es el proceso de revertir un sistema, aplicación o base de datos a un estado estable anterior después de una implementación fallida o un error. Sirve como mecanismo de seguridad, asegurando la integridad de los datos y minimizando el tiempo de inactividad al deshacer los cambios defectuosos para restaurar la funcionalidad.

Despliegue



Artefacto:

Notas de lanzamiento

○ Release notes

Documentan qué cambió en esta versión del sistema. La documentación de usuario explica cómo operar el sistema en lenguaje accesible para usuarios no técnicos.

Errores típicos:

- Despliegue sin plan de rollback: si algo falla no hay forma rápida de volver atrás
- Migración de datos sin prueba previa: se ejecuta directo en producción sin validar en staging
- Capacitación insuficiente: el sistema funciona pero los usuarios no saben operarlo
- Despliegue sin monitoreo: nadie está mirando el sistema en las primeras horas críticas

Desplegar no es el final del trabajo. Es el momento en que el trabajo se enfrenta a la realidad.

Mantenimiento



¿Qué se hace?	¿Quién lo hace?
Se corrigen bugs reportados, se adapta el sistema cuando la clínica suma un segundo médico y se añade una funcionalidad de turnos por especialidad, etc.	Equipo de desarrollo, con input de la recepcionista y el médico.
¿Qué produce?	¿Qué pasa si se hace mal?
No hay un artefacto único. Cada cambio genera una nueva versión del sistema.	Se agrega el segundo médico modificando código sin documentar. Tres meses después nadie sabe por qué el sistema a veces duplica turnos.

¿De qué se trata?

Mantener el sistema funcionando y alineado con las necesidades cambiantes del negocio. No tiene fecha de cierre.

¿Qué se hace?

- Corrección de bugs que aparecen en producción
- Adaptación a cambios externos: nueva legislación, nuevo hardware, nuevas integraciones
- Incorporación de nuevas funcionalidades solicitadas por los usuarios
- Optimización de performance y seguridad
- Gestión de la deuda técnica acumulada

Mantenimiento



Artefacto:

Nuevas versiones del sistema

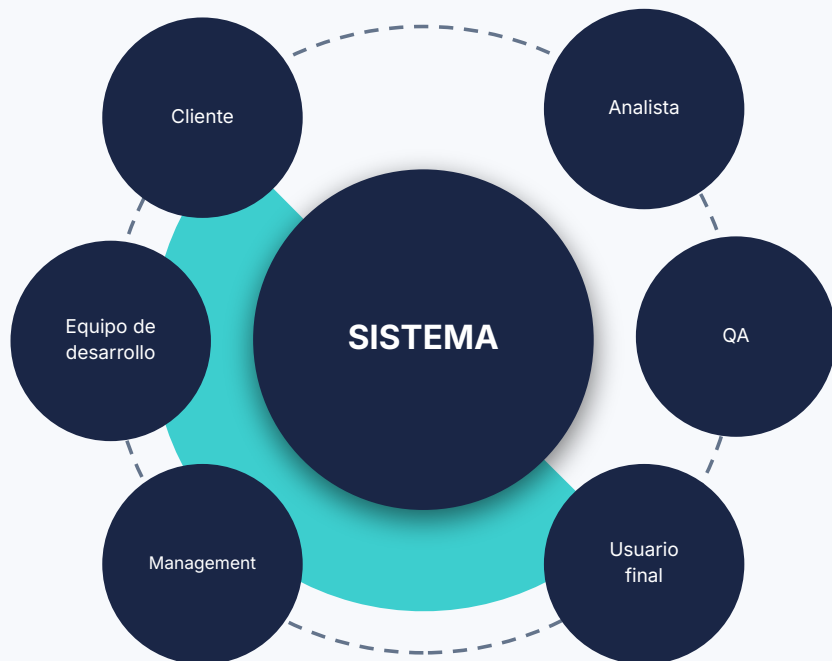
Cada ciclo de mantenimiento produce una nueva versión: correcciones, adaptaciones o mejoras documentadas en release notes y registradas en el historial de cambios del sistema.

Errores típicos:

- Corregir sin entender: se parchea el síntoma sin resolver la causa raíz
- Sin control de versiones: nadie sabe qué cambió, cuándo y por qué
- Deuda técnica ignorada: se agregan funcionalidades sobre una base frágil sin sanearla
- Sin documentación actualizada: el sistema evoluciona pero la documentación queda desactualizada

El mantenimiento no es el final del ciclo de vida. Es la etapa más larga.

Stakeholders



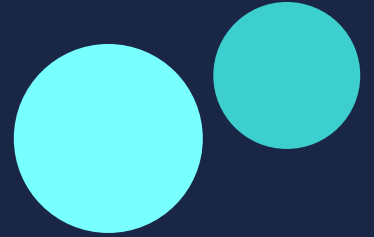
Parte interesada (Stakeholder)

Cualquier individuo, grupo u organización que puede afectar, verse afectado o percibirse a sí mismo como afectado por las decisiones, actividades o resultados de un proyecto.

Stakeholder	Interés principal
Cliente	Que el sistema resuelva su problema al menor costo
Usuario final	Que sea fácil de usar y no interrumpa su trabajo
Analista	Que los requerimientos sean claros y completos
Equipo de desarrollo	Que el diseño sea sólido y el alcance esté definido
QA	Que haya tiempo suficiente para probar correctamente
Management	Que se entregue en tiempo y dentro del presupuesto

Clase 4

Requerimientos I



Requerimientos



Recordemos: "La mayor parte de los problemas en proyectos de software se originan en requerimientos mal definidos."

¿En qué nos vamos a enfocar?

1. ¿Qué necesita exactamente el cliente?
2. ¿Cómo lo obtenemos?
3. ¿Cómo lo documentamos para que el equipo pueda construirlo?

Requerimientos funcionales

¿Qué son? Describen el comportamiento del sistema: qué funciones debe realizar, qué datos debe procesar y qué reglas de negocio debe respetar.

Un requerimiento funcional siempre describe un comportamiento observable: algo que el sistema hace, produce o permite hacer.

No describe cómo lo hace internamente, eso es diseño. Describe qué hace desde la perspectiva del usuario.

Ejemplos con el sistema de turnos médicos:

#	Requerimiento funcional
1	El sistema debe permitir registrar un nuevo paciente con nombre, DNI y teléfono
2	El sistema debe permitir asignar un turno a un paciente para un médico en una fecha y hora específica
3	El sistema debe enviar un recordatorio al paciente 24 horas antes del turno
4	El sistema debe permitir cancelar un turno con al menos 2 horas de anticipación
5	El sistema debe mostrar la agenda diaria del médico ordenada por hora

Antes de avanzar, una nueva definición:

Funcionalidad (Feature)

Una funcionalidad de software es una característica o capacidad específica, visible para el usuario, de una aplicación, diseñada para resolver una necesidad concreta del usuario o aportar valor al negocio.

Es una unidad, a menudo, aislable, como una barra de búsqueda, la autenticación de usuarios o un botón para activar el “modo oscuro”.

Funcionalidades vs requerimientos funcionales

- Una funcionalidad es una capacidad del sistema de alto nivel como "gestión de turnos"
- Un requerimiento funcional es la descomposición de esa funcionalidad en comportamientos específicos, concretos y verificables.
- La diferencia importa porque una funcionalidad no se puede probar directamente, un requerimiento funcional sí.

Ejemplos con el sistema de turnos médicos:

Funcionalidad		Requerimientos funcionales
Gestión de turnos	1	El sistema debe permitir asignar un turno a un paciente para un médico en una fecha y hora específica.
	2	El sistema debe permitir cancelar un turno con al menos 2 horas de anticipación.
	3	El sistema debe mostrar la agenda diaria del médico ordenada por hora.
Gestión de pacientes	4	El sistema debe permitir registrar un nuevo paciente con nombre, DNI y teléfono.
	5	El sistema debe permitir buscar un paciente por DNI o por nombre.
Notificaciones	6	El sistema debe enviar un recordatorio al paciente 24 horas antes del turno.

Una funcionalidad no se puede construir ni probar. Un requerimiento funcional, sí.

Requerimientos no funcionales

¿Qué son? Describen las cualidades del sistema: no qué hace, sino cómo lo hace y bajo qué condiciones opera correctamente.

- Los requerimientos no funcionales son tan importantes como los funcionales.
- Frecuentemente se omiten o se escriben de forma vaga.

Ejemplos con el sistema de turnos médicos:

Categoría	Ejemplo
Performance	El sistema debe responder en menos de 2 segundos con hasta 100 usuarios simultáneos
Seguridad	Solo el médico asignado puede ver la historia clínica de sus pacientes
Usabilidad	Un recepcionista sin capacitación técnica debe poder registrar un turno en menos de 3 pasos
Escalabilidad	El sistema debe soportar hasta 10 clínicas sin cambios de arquitectura
Disponibilidad	El sistema debe estar disponible el 99% del tiempo en horario de atención

Comparemos estos 2 requerimientos:

- 1) "El sistema debe ser rápido" → No podemos comparar, medir ni verificar.
- 2) "El sistema debe responder en menos de 2 segundos con hasta 100 usuarios simultáneos" → Descripción específica. Podemos medirlo y verificarlo.

Los requerimientos no funcionales tienen un impacto directo en las decisiones de arquitectura y diseño

Requerimientos de usuario vs. de sistema

Requerimiento de usuario	Requerimiento de sistema
El recepcionista debe poder registrar un nuevo turno	El sistema debe validar que la fecha y hora del turno no coincida con otro turno existente para el mismo médico antes de confirmar el registro
El paciente debe poder cancelar su turno	El sistema debe permitir la cancelación de un turno hasta 2 horas antes de la hora asignada, actualizando el estado del turno a "cancelado" y liberando el horario en la agenda del médico
El médico debe poder ver su agenda	El sistema debe mostrar los turnos del día ordenados por hora, con nombre del paciente, motivo de consulta y estado del turno (confirmado, cancelado, pendiente)

El requerimiento de usuario describe qué necesita el usuario. El de sistema describe qué tiene que hacer el software para satisfacer esa necesidad.

- Los requerimientos de usuario están **escritos en lenguaje del negocio, sin tecnicismos**, para que el cliente y los usuarios finales los puedan leer, entender y validar.
- Los requerimientos de sistema están **escritos en lenguaje técnico, con precisión** suficiente para que el equipo de desarrollo pueda implementarlos **sin ambigüedad**.

Requerimientos de usuario vs. de sistema: el analista

- El analista toma una necesidad expresada en lenguaje del negocio y la traduce a especificaciones técnicas.
- En esa traducción puede agregar restricciones que el cliente no pidió, omitir casos que el cliente daba por obvios o interpretar de forma distinta lo que el cliente quería decir.
- La forma de mitigar ese riesgo es mostrarle los requerimientos de sistema al cliente antes de arrancar el diseño y confirmar que la traducción es correcta.
- Se busca que el cliente confirme que el comportamiento descrito es el que necesita.



Cómo escribir un requerimiento bien definido

¿Qué elementos tiene un requerimiento bien definido?

- **Sujeto claro:** quién o qué realiza la acción.
- **Verbo preciso:** debe, no debería ni podría.
- **Condición específica:** bajo qué circunstancias.
- **Criterio verificable:** cómo sabemos que está cumplido.

Comparación:

Mal escrito	Bien escrito
El sistema debe ser rápido	El sistema debe responder en menos de 2 segundos con hasta 100 usuarios simultáneos.
El usuario puede cancelar turnos	El sistema debe permitir cancelar un turno hasta 2 horas antes de la hora asignada.
El sistema debe ser seguro	Solo el médico asignado puede acceder a la historia clínica de sus pacientes.

Un requerimiento bien escrito no deja lugar a interpretación.

Técnicas de relevamiento: entrevistas

¿Qué es?

Conversación estructurada entre el analista y uno o más stakeholders con el objetivo de obtener información sobre necesidades, procesos y restricciones.

- La regla de documentar textualmente durante la entrevista es crítica.
- El analista que interpreta en tiempo real está introduciendo sus propios supuestos en el registro.
- Lo que el cliente dijo exactamente y lo que el analista entendió pueden ser cosas distintas.

2 tipos de entrevista:

Tipo	Cuándo usarla
Estructurada	El analista tiene un listado de preguntas predefinidas. Útil cuando el tiempo es limitado o el stakeholder es difícil de acceder.
Semi-estructurada	El analista tiene un guión base pero permite que la conversación derive hacia temas no anticipados. Útil para descubrir requerimientos implícitos.

Reglas prácticas:

- Entrevistar al usuario final, no solo al cliente que paga o gestiona el proyecto.
- Preguntar por el proceso actual antes de hablar del sistema nuevo
- Documentar textualmente, no interpretar durante la entrevista

Técnicas de relevamiento: observación directa y talleres conjuntos

Observación

El analista observa al usuario trabajando en su entorno real sin interrumpirlo. Detecta requerimientos implícitos que el usuario nunca mencionaría en una entrevista porque los considera obvios.

Cuándo usarla:

Cuando **el cliente tiene dificultad para describir su proceso** o cuando se sospecha que hay requerimientos implícitos no declarados.

Talleres conjuntos

Sesión estructurada de varias horas donde todos los stakeholders relevantes se reúnen al mismo tiempo para construir requerimientos en conjunto y resolver conflictos en el momento.

Cuándo usarla:

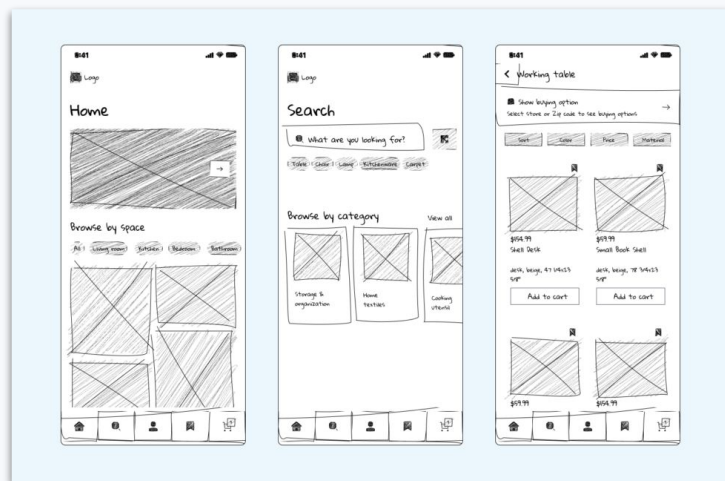
Cuando **hay múltiples stakeholders con intereses distintos** o cuando las entrevistas individuales generaron **requerimientos contradictorios**.

Técnicas de relevamiento: prototipado

Prototipado de baja fidelidad

Bocetos en papel o wireframes simples sin funcionalidad real.

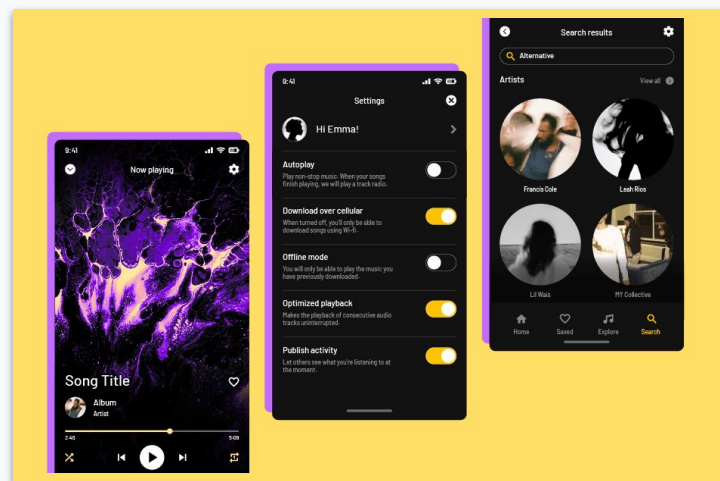
Cuando usarlo? Al inicio del relevamiento para validar flujos y pantallas.



Prototipado de alta fidelidad

Maqueta interactiva que simula la experiencia real del sistema.

Cuándo usarlo? Cuando los requerimientos están más avanzados y se necesita confirmación detallada.



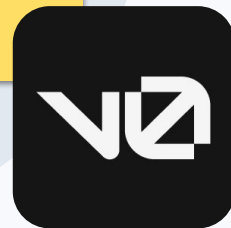
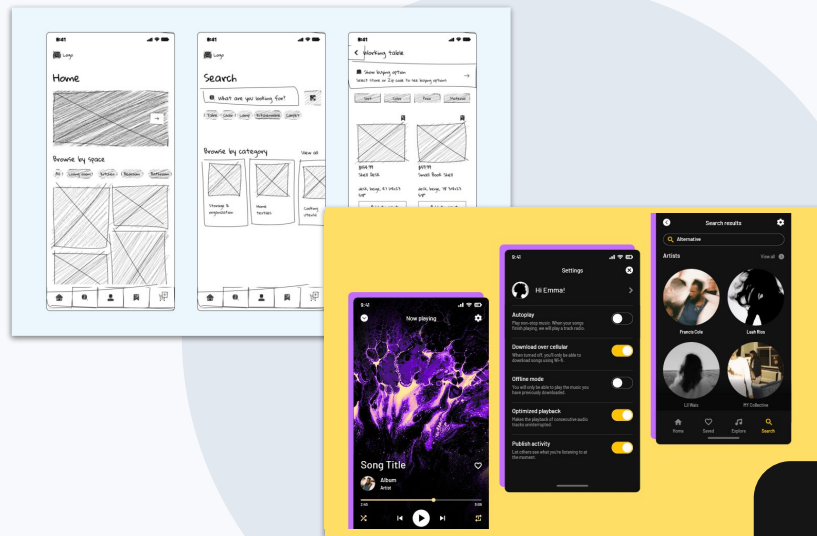
Técnicas de relevamiento: prototipado

El valor real del prototipado es que **hace visible lo que las palabras no pueden describir con precisión.**

Un cliente puede tener dificultad para expresar cómo quiere que se vea una pantalla, pero cuando se le muestra un boceto puede decir inmediatamente cosas como:

"esto no es lo que necesito", "falta este dato".

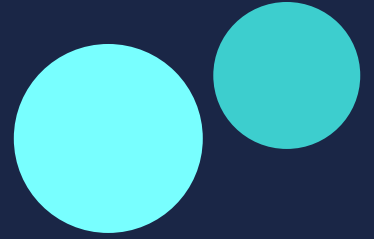
Esa **retroalimentación temprana** es mucho más barata que descubrirlo durante el testing o después del despliegue.



El prototipo es una herramienta de comunicación y validación, no un punto de partida para el desarrollo.

Problemas frecuentes de la ingeniería de requerimientos

Problema	Qué es	Cómo resolverlo
Ambigüedad	Un requerimiento que dos personas interpretan distinto.	Agregar criterios de verificación concretos y medibles.
Implícitos	Algo que el cliente asume que es obvio y no menciona.	Observación del proceso actual y revisión con checklist.
Contradictorios	Dos stakeholders piden cosas incompatibles.	Taller conjunto para resolver el conflicto con todos presentes.
Alcance sin control (Scope creep)	El alcance crece durante el relevamiento sin control.	Proceso formal de change request para cualquier agregado.



Clase 5

Requerimientos II

Historias de usuario

El requerimiento desde la perspectiva del usuario

¿Qué es?

Una descripción corta y simple de una funcionalidad del sistema desde la perspectiva del usuario que la necesita.

No describe cómo se implementa, describe qué valor genera para quién.

Estructura:

"Como [rol], quiero [acción], para [valor/beneficio]."

¿Para qué sirve?

- Poner el foco en el valor al usuario, no en la funcionalidad técnica.
- Facilitar la conversación entre el cliente y el equipo.
- Ser la unidad básica de planificación en metodologías ágiles.

Componente	Pregunta que responde	Por qué importa
Rol	¿Quién necesita esto?	Define la perspectiva y el contexto de uso
Acción	¿Qué quiere hacer?	Define el comportamiento esperado del sistema
Valor	¿Para qué lo necesita?	Define la intención detrás de la funcionalidad

Criterios de aceptación

¿Qué son?

Condiciones concretas y verificables que deben cumplirse **para considerar una historia de usuario correctamente implementada**. Son el contrato entre el equipo y el cliente sobre qué significa "listo".

Sin criterios de aceptación, nadie sabe cuándo una historia está realmente terminada.

Historia de usuario	Criterios de aceptación
Como recepcionista , quiero registrar un nuevo turno , para confirmar la atención del paciente .	- El sistema valida que el horario no esté ocupado. - El paciente recibe confirmación por mensaje. - El turno aparece en la agenda del médico.
Como paciente , quiero cancelar mi turno , para liberar el horario si no puedo asistir .	- Solo se puede cancelar con más de 2 horas de anticipación. - El horario queda disponible para otro paciente. - El paciente recibe confirmación de la cancelación.
Como médico , quiero ver mi agenda del día , para organizarme antes de empezar a atender .	- Los turnos se muestran ordenados por hora. - Se muestra nombre del paciente y motivo de consulta. - Se diferencia visualmente entre turnos confirmados y cancelados.

Historias de usuario aplicadas al ejemplo de nuestra app

Rol	Historia completa	Criterios de aceptación
Recepcionista	Como recepcionista, quiero registrar un nuevo paciente, para tenerlo disponible al momento de asignarle un turno.	- Se requiere nombre, DNI y teléfono como mínimo. - El sistema valida que el DNI no esté registrado previamente. - El paciente queda disponible para asignar turnos inmediatamente.
Paciente	Como paciente, quiero ver los turnos disponibles de un médico, para elegir el horario que más me convenga.	- Se muestran solo horarios futuros - Se filtra por especialidad y médico. - Los horarios ocupados no aparecen en la lista.
Médico	Como médico, quiero ver mi agenda del día, para organizarme antes de empezar a atender.	- Los turnos se muestran ordenados por hora. - Se muestra nombre del paciente y motivo de consulta. - Se diferencia visualmente entre turnos confirmados y cancelados.
Paciente	Como paciente, quiero cancelar mi turno, para liberar el horario si no puedo asistir.	- Solo se puede cancelar con más de 2 horas de anticipación. - El horario queda disponible para otro paciente. - El paciente recibe confirmación de la cancelación.
Administrador	Como administrador, quiero ver cuántos turnos se atendieron por día, para evaluar la ocupación de la clínica.	- Se puede filtrar por médico y por rango de fechas. - Se muestra el total de turnos confirmados, cancelados y atendidos. - Los datos se pueden exportar.

Antes de avanzar, una nueva definición:

Flujo feliz (Happy Path)

Escenario ideal por defecto en el que un usuario completa una tarea con éxito, sin encontrar errores, excepciones ni comportamientos inesperados. Representa el recorrido más directo y libre de fricciones.

Casos de uso

El requerimiento desde la perspectiva del sistema

¿Qué es?

Una descripción detallada y estructurada de cómo el sistema responde a una acción de un actor externo.

Documenta no solo el flujo exitoso o *happy path* sino todas las variaciones y errores posibles.

Los casos de uso y las historias de usuario son herramientas complementarias, cada una con un distinto nivel de profundidad.

Debilidades de la historia de usuario.

Las historias de usuario son simples, centradas en el valor y fáciles de escribir.

Pero tienen un límite: no describen qué pasa cuando algo sale mal.

No documentan los flujos alternativos ni las excepciones.

Cuando una funcionalidad es suficientemente compleja como para tener múltiples flujos posibles y varios puntos de falla, un caso de uso completo es la forma más efectiva de documentarla.

Casos de uso

El requerimiento desde la perspectiva del sistema

Componente	Descripción
Actor	Quién interactúa con el sistema: recepcionista, médico, paciente.
Precondición	Qué debe ser verdad antes de que el caso de uso pueda ejecutarse.
Flujo principal	La secuencia de pasos del caso exitoso.
Flujos alternativos	Variaciones válidas del flujo principal (<i>happy path</i>).
Excepciones	Qué pasa cuando algo sale mal.

Documenta no sólo el flujo exitoso sino también las variaciones y los errores.

Casos de uso aplicado a nuestra app

Ejemplo: Sacar un turno.

Actor: Paciente

Precondición: El paciente existe en el sistema y el médico tiene disponibilidad en la fecha seleccionada.

Flujo principal:

1. El paciente selecciona la opción "Sacar turno".
2. El sistema muestra las especialidades disponibles.
3. El paciente selecciona una especialidad.
4. El sistema muestra los médicos disponibles para esa especialidad.
5. El paciente selecciona un médico.
6. El sistema muestra los horarios disponibles.
7. El paciente selecciona fecha y hora.
8. El sistema confirma el turno y envía una notificación al paciente.

Flujo alternativo:

El paciente no encuentra un horario conveniente: puede volver al paso 4 y seleccionar otro médico de la misma especialidad.

Excepciones:

- El horario seleccionado ya no está disponible: el sistema muestra un mensaje de error y vuelve al paso 6.
- El sistema no puede enviar la notificación: el turno se confirma igual y se registra el error de notificación.

Historias de usuario vs casos de uso



Validación de requerimientos

¿Cómo sabemos que los requerimientos están bien antes de diseñar?

Revisión estructurada

El analista y al menos otro miembro del equipo recorren el documento de requerimientos línea por línea buscando ambigüedades, contradicciones e incompletitudes. Se documenta cada problema encontrado y se resuelve antes de avanzar.

Presentación al cliente

El analista presenta los requerimientos al cliente y los usuarios finales en lenguaje accesible. El objetivo es confirmar que la traducción del problema original al documento de requerimientos es correcta. No es una reunión de aprobación formal, es una conversación de verificación.

Prototipo de validación

Se construye un prototipo de baja o alta fidelidad y se lo presenta al cliente para confirmar que el comportamiento esperado del sistema coincide con lo documentado. Es la técnica más efectiva para detectar requerimientos implícitos que sobrevivieron al relevamiento.

La etapa de validación de requerimientos es la última oportunidad barata de detectar errores.

Trazabilidad de requerimientos

¿Cómo sabemos que los requerimientos se van a mantener durante el resto de ciclo de vida del desarrollo?



Requerimiento	Diseño	Implementación	Prueba
RF02: El sistema debe permitir asignar un turno	Modelo de datos: tabla turnos con campos médico, paciente, fecha, hora	Función crearTurno() en el módulo de agenda	Caso de prueba CP05: verificar que un turno se crea correctamente
RF04: El sistema debe permitir cancelar un turno con al menos 2 horas de anticipación	Regla de negocio en la capa de lógica	Función cancelarTurno() con validación de tiempo	Caso de prueba CP08: verificar que no se puede cancelar con menos de 2 horas

Si un requerimiento no tiene diseño, implementación y prueba asociados, algo se perdió en el camino.

Actividad

Caso: Expansión del sistema de turnos médicos

La clínica que usa el sistema de turnos incorporó tres médicos nuevos de distintas especialidades y ahora quiere que los pacientes puedan sacar turnos directamente desde WhatsApp. Además, el administrador de la clínica necesita saber cuántos turnos se atienden por día, cuántos se cancelan y cuáles son los médicos con más demanda.

Consignas:

1. Identificar al menos 4 requerimientos funcionales y 2 no funcionales.
2. Escribí 2 historias de usuario con sus criterios de aceptación.
3. Desarrollá el caso de uso completo para "Sacar turno desde WhatsApp".
4. Identificá al menos 2 requerimientos implícitos que el enunciado no menciona explícitamente.

Actividad

1) a. Requisitos funcionales:

- El sistema debe permitir al paciente consultar turnos disponibles por especialidad a través de WhatsApp
- El sistema debe permitir al paciente confirmar un turno desde WhatsApp sin intervención de la recepcionista
- El sistema debe permitir al paciente cancelar un turno desde WhatsApp con al menos 2 horas de anticipación
- El sistema debe mostrar al administrador un reporte diario con turnos atendidos, cancelados y médicos con más demanda

1) b. No funcionales:

- El sistema debe responder a los mensajes de WhatsApp en menos de 5 segundos
- La integración con WhatsApp debe cumplir con la API oficial de WhatsApp Business

2) Historias de usuario y criterios de aceptación

Historia de usuario	Criterios de aceptación
Como paciente , quiero consultar turnos disponibles por WhatsApp , para elegir el horario que más me convenga sin llamar a la clínica .	- El sistema responde con una lista de especialidades disponibles - Al seleccionar especialidad muestra médicos disponibles - Al seleccionar médico muestra fechas y horarios disponibles
Como administrador , quiero ver un reporte diario de turnos , para identificar los médicos con más demanda y tomar decisiones operativas .	- El reporte muestra cantidad de turnos atendidos y cancelados por día - Muestra ranking de médicos ordenado por cantidad de turnos - Se puede filtrar por especialidad y por rango de fechas

Actividad

3) Caso de uso: Sacar turno desde WhatsApp

- **Actor:** Paciente
- **Precondición:** El paciente tiene WhatsApp y el número de la clínica está registrado como contacto
- **Flujo principal:** El paciente envía un mensaje al número de WhatsApp de la clínica. El sistema responde con un menú de especialidades. El paciente selecciona una especialidad. El sistema muestra los médicos disponibles. El paciente selecciona un médico. El sistema muestra los turnos disponibles. El paciente selecciona fecha y hora. El sistema confirma el turno y envía un mensaje de confirmación con los datos del turno.
- **Flujo alternativo:** El paciente no está registrado en el sistema. El sistema solicita nombre y DNI antes de continuar con la selección de turno.
- **Excepción:** El turno seleccionado ya no está disponible. El sistema informa al paciente y le solicita seleccionar otro horario.

4) Requerimientos implícitos:

- El paciente debe poder identificarse antes de sacar un turno para que el sistema asocie el turno a su historia clínica. El enunciado no menciona autenticación pero es un requisito obvio de seguridad y operación.
- El sistema debe enviar un recordatorio por WhatsApp antes del turno. Cualquier paciente que saca un turno por WhatsApp va a asumir que el recordatorio llega por el mismo canal.